

INTELLIGENT SYSTEM DEVELOPMENT

Dinh Que Tran, Ph.D., Assoc. Prof.

Posts and Telecommunications Institute of Technology (PTIT)

Mail: quedt@ptit.edu.vn, tdque@yahoo.com

Lecture 01

From Intelligence to Intelligent Systems

Understand intelligence, representations, learning, reasoning, decision, action, and system development.

Course theme

Understand → Represent → Learn → Decide → Act → Adapt

Intelligence System Development: A General View

Dimension	Key Components	Central Questions
Intelligence	▶ Perception ▶ Representation ▶ Learning ▶ Reasoning ▶ Decision ▶ Action/Reflection	▶ What does the system know? ▶ How does it learn? ▶ How does it reason? ▶ How does it decide?
System	▶ Environment ▶ Data ▶ Internal State ▶ Model ▶ Interface ▶ Memory/Feedback	▶ What does the system observe? ▶ What does it represent? ▶ How does it interact? ▶ How does it adapt?

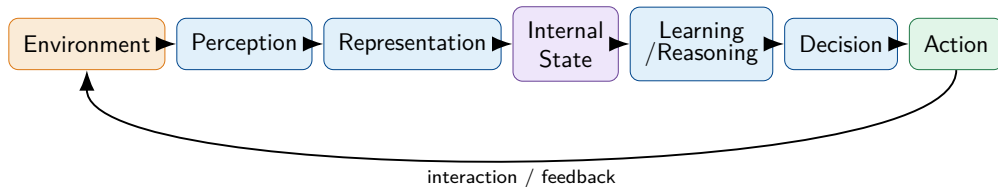
Intelligence System Development: A General View (cont)

Dimension	Key Components	Central Questions
Development	▶ Understand ▶ Represent ▶ Implement ▶ Experiment ▶ Evaluate ▶ Deploy ▶ Iterate	▶ How do we build it? ▶ How do we validate it? ▶ How do we deploy it? ▶ How do we improve it?

Intelligence + System + Development $\Rightarrow$ Intelligent System

Why Do We Study Intelligent System Development?

An intelligent system is more than a trained machine-learning model.

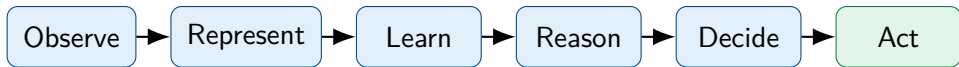


- ▶ AI methods provide mechanisms for computation and learning.
- ▶ An intelligent system connects those mechanisms to an environment.
- ▶ Development therefore includes data, representation, models, experiments, interfaces, deployment, feedback, and evaluation.

ML model $\neq$ complete intelligent system

What Is Intelligence?

There is no single universally accepted engineering definition.
For this course, we use a functional view:



Intelligence can be viewed through capabilities such as:

- ▶ acquiring information from the environment;
- ▶ forming useful internal representations;
- ▶ learning from data or experience;
- ▶ reasoning under uncertainty or constraints;
- ▶ making goal-directed decisions;
- ▶ acting and adapting from feedback.

Intelligence Is Not One Algorithm

Different forms of intelligent behavior may require different mechanisms.

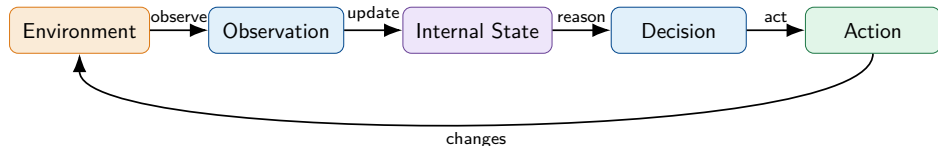
Capability	Possible mechanism	Example
Perception	Computer vision	Image understanding
Representation	Features / embeddings / graphs	Semantic state
Learning	ML / deep learning	Prediction
Reasoning	Logic / search / inference	Planning
Decision	Optimization / policy	Action selection
Interaction	Dialogue / agents	Conversational system
Adaptation	Feedback / continual learning	Changing environment

Intelligence emerges from coordinated capabilities.

What Is an Intelligent System?

A useful engineering abstraction is:

Intel Syst = (Environment, State, Representation, Knowledge, Learning, Decision, Action)

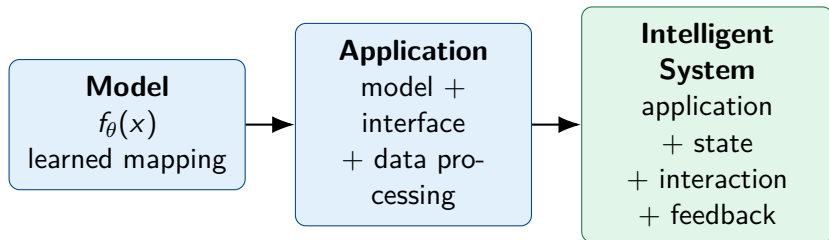


The key distinction is between:

external state and internal / cognitive state.

The system does not directly possess the whole environment. It operates through observations and representations.

Model, Application, and Intelligent System



- ▶ A **model** computes a learned function.
- ▶ An **application** makes the model usable.
- ▶ An **intelligent system** operates in an environment and can use state, knowledge, interaction, and feedback.

Development must address the whole system, not only the model.

A Brief History of Artificial Intelligence

The field has evolved through several major paradigms.

Symbolic → Expert Systems → Statistical ML → Deep Learning → Foundation Models → Interactive/Agentic AI

The important question is not only:

Which algorithm is newer?

but:

What changed in representation, learning, and system architecture?

AI Development: A Conceptual Timeline

Period	Paradigm	Main idea	Typical representation
1950s–60s	Symbolic AI	Logic, search, problem solving	Symbols / rules
1970s–80s	Expert systems	Encode domain expertise	Rules / knowledge base
1980s–2000s	Statistical ML	Learn patterns from examples	Feature vectors

AI Development: A Conceptual Timeline (cont)

Period	Paradigm	Main idea	Typical representation
2000s–2010s	Deep learning	Learn hierarchical representations	Tensors
2010s	Large-scale deep learning	Data + compute + learned representations	High-dimensional tensors
Late 2010s–2020s	Transformers	Attention-based sequence modeling	Embeddings / tokens
2020s	Foundation / generative AI	General-purpose learned representations	Multimodal embeddings
Current direction	Interactive / agentic systems	Perceive, reason, use tools, act	State + memory + tools

Era 1: Symbolic AI

A central idea was to represent knowledge explicitly.

Knowledge $\rightarrow$ Symbols $\rightarrow$ Rules $\rightarrow$ Inference

Example:

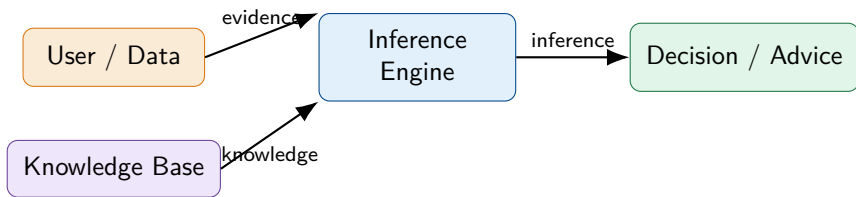
IF fever AND cough
THEN possible_infection.

- ▶ Knowledge is explicit.
- ▶ Reasoning can be inspected.
- ▶ Search and logic are important.
- ▶ Systems can be brittle when knowledge is incomplete.

Strength: explicit reasoning	Limitation: knowledge acquisition
------------------------------	-----------------------------------

Era 2: Expert Systems

Expert systems made symbolic AI practical in many domains.



The central development problem became:

How do we acquire, represent, maintain, and use knowledge?

This question remains important today in knowledge graphs, retrieval systems, and AI agents.

Era 3: Statistical Machine Learning

A major shift:

Program the rules $\longrightarrow$ Learn patterns from examples

Given $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, learn $\hat{y} = f_{\theta}(x)$.

Typical methods include:

- ▶ Logistic Regression
- ▶ k-Nearest Neighbors
- ▶ Support Vector Machines
- ▶ Decision Trees
- ▶ Random Forests

Representation becomes a central engineering problem.

Era 4: Deep Learning

Traditional ML often uses:

Data $\rightarrow$ Human-designed features $\rightarrow$ ML model.

Deep learning moves toward:

Data $\rightarrow$ Learned representation $\rightarrow$ Prediction

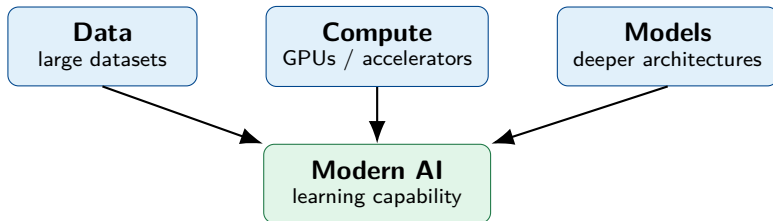
A neural network can learn multiple levels of representation:

$$x \rightarrow h_1 \rightarrow h_2 \rightarrow \cdots \rightarrow h_L \rightarrow y.$$

Representation learning becomes part of the model.

Why Did Deep Learning Become So Powerful?

Three reinforcing factors became particularly important:



The development pattern is important:

More data + more compute + better architectures $\Rightarrow$ new capabilities

This is one reason intelligent-system development is also a systems-engineering problem.

Era 5: Transformers and Attention

A major architectural development was the Transformer, introduced in 2017.
Its key mechanism is attention.

Conceptually:

tokens $\rightarrow$ embeddings $\rightarrow$ attention $\rightarrow$ contextual representations.

A simplified attention expression is:

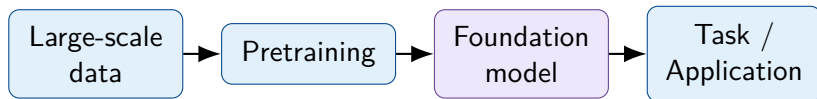
$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V.$$

The important course-level idea is:

Context can be represented and learned at scale.

Era 6: Foundation and Generative AI

Large pretrained models can provide reusable capabilities.



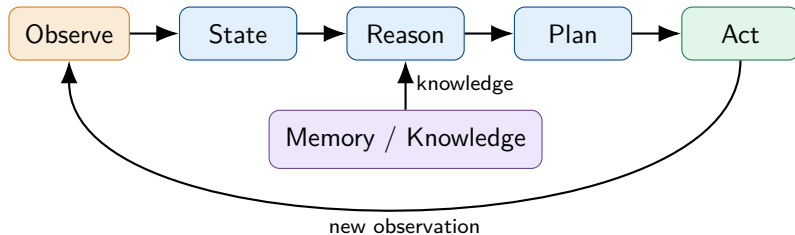
Examples of capabilities include:

- ▶ language understanding and generation;
- ▶ image and multimodal understanding;
- ▶ code generation;
- ▶ retrieval and question answering;
- ▶ adaptation to many downstream tasks.

One model can support many applications.

Current Direction: Interactive and Agentic Systems

A modern intelligent system increasingly does more than generate an output.



The conceptual structure is now:

Observe → State → Reason → Plan → Act → Observe

with

Memory / Knowledge → Reasoning

This is a nice ****agentic intelligence**** diagram: the system maintains a state, reasons using memory/knowledge, plans, acts, and then learns from the resulting new observations.

Current Direction: Interactive and Agentic Systems (cont)

This introduces important system concepts:

state + memory + tools + interaction + feedback

What Really Changed Through AI History?

The evolution can be summarized along four dimensions.

	Knowledge	Representation	Learning
Symbolic AI	explicit	symbols	limited / programmed
Expert systems	explicit	rules	knowledge engineering
Statistical ML	data-driven	features	statistical learning
Deep learning	mostly learned	tensors	representation learning
Foundation models	pretrained	embeddings	large-scale pretraining
Interactive AI	external + learned	state + tools	continual interaction

The history of AI is also a history of representations.

Evolution of Representation

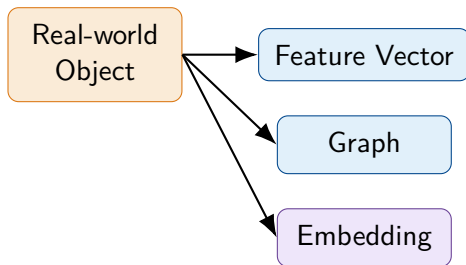


Different information structures naturally suggest different representations:

- ▶ records $\rightarrow$ feature vectors;
- ▶ images $\rightarrow$ tensors;
- ▶ relations $\rightarrow$ graphs;
- ▶ language $\rightarrow$ tokens and embeddings;
- ▶ multiple modalities $\rightarrow$ integrated representations.

Representation Is Not a Neutral Choice

Consider the same real-world object.



Each representation preserves some information and discards or transforms other information.

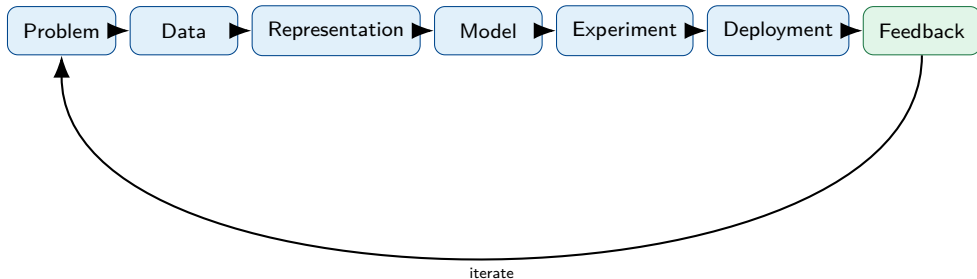
Therefore:

Representation $\rightarrow$ available information $\rightarrow$ possible learning

This principle will reappear throughout the course.

What Does “Development” Mean?

Development is not simply writing code.



What Does “Development” Mean?

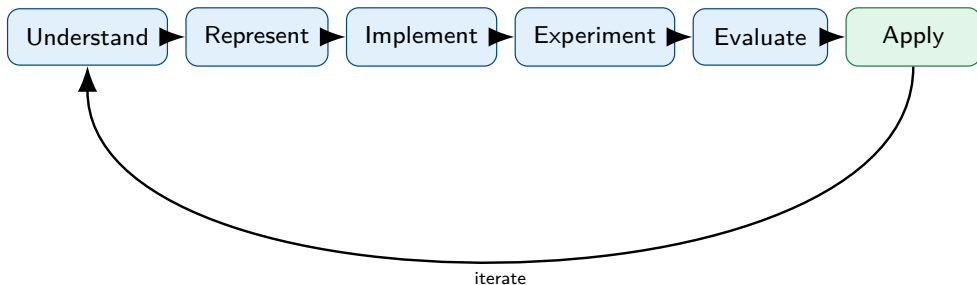
Development is not simply writing code. A development process must answer:

- ▶ What problem are we solving?
- ▶ What information is available?
- ▶ How should it be represented?
- ▶ What intelligence mechanism is appropriate?
- ▶ How do we evaluate it?
- ▶ How does it become a reliable application?

The Course Development Loop

We will repeatedly use:

Understand → Represent → Implement → Experiment → Evaluate → Apply



A good intelligent system is developed through evidence and iteration.

From Data to Intelligent Behavior

A useful abstraction for the whole course is:

Data → Representation → Learning → Knowledge → Decision → Action

But the process is not necessarily one-way.

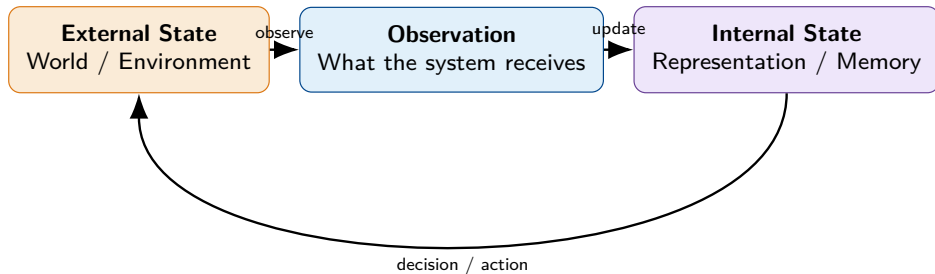
Action → Feedback → New Data → Updated System.

This creates a development loop:

Observe → Learn → Act → Observe again

External State and Internal State

An intelligent system interacts with an environment.



Important distinction:

internal state $\neq$ external state

The internal state is the system's computational representation of what it currently knows, remembers, or believes.

Adaptation: Intelligence Under Change

An intelligent system may operate in a changing environment.

$$E_t \rightarrow o_t \rightarrow s_t \rightarrow a_t \rightarrow E_{t+1}.$$

Here:

- ▶ E_t : environment at time t ;
- ▶ o_t : observation;
- ▶ s_t : internal state;
- ▶ a_t : action.

The system may update its internal state:

$$s_{t+1} = F(s_t, o_t, a_t).$$

Intelligence becomes dynamic when state and feedback matter.
--

The Central Question of This Course

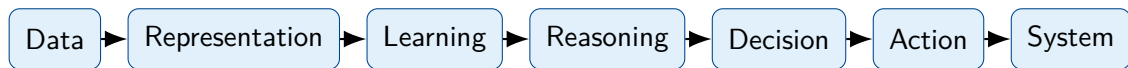
Instead of asking only:

How do I train a model?

we ask:

How do I develop a system that can use intelligence?

That requires understanding:



Course Roadmap

Stage	Main representation	Main capability
A1	Feature vectors	Traditional ML + deployment
A2	Data / improved representations	Preprocessing + evaluation
A3	Learned representations	Neural networks + deep learning
A4	Tensors + graphs	CNN + relational knowledge
A5	Embeddings + context	RAG + conversational systems
Project	Integrated representations	Complete intelligent system

Each assignment adds a new capability.

Assignment 01: Where We Begin

We begin with a relatively simple setting:

Structured data $\rightarrow$ Feature vector $\rightarrow$ Traditional ML $\rightarrow$ Application

Why start here?

- ▶ The mathematical formulation is transparent.
- ▶ Students can inspect the representation directly.
- ▶ Several classical algorithms can be compared.
- ▶ Experiments can be controlled and explained.
- ▶ Deployment introduces the system perspective.

We start simple so that the development principles are visible.

The Philosophy of the Course



We do NOT want students to:

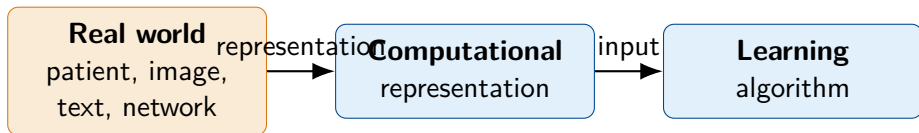
- ▶ only call `fit()`;
- ▶ copy a notebook;
- ▶ report one accuracy value;
- ▶ choose a model without evidence.

We want students to:

- ▶ understand the representation;
- ▶ understand the algorithm;
- ▶ design experiments;
- ▶ explain the results;
- ▶ build a usable application.

The Central Representation Question

What does a learning algorithm actually receive?



Central Principle

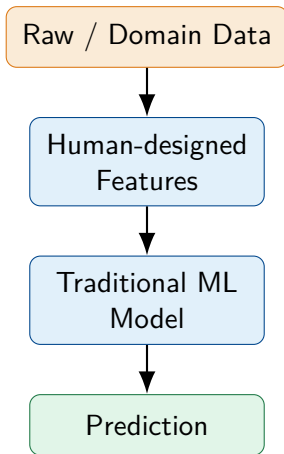
The representation determines what information is available to the learner.

Representation Is the Backbone

Information	Representation	Typical models	Later topic
Structured records	Feature vector $x \in \mathbb{R}^d$	LR, KNN, SVM, Trees	A1–A2
Image	Tensor	CNN, ViT	Deep Learning
Sequence	Vector sequence	RNN, LSTM, Transformer	Deep Learning
Graph	Nodes + edges	GNN	Knowledge / Graph
Knowledge	Entities + relations	KG reasoning	Knowledge Graph
Documents	Embeddings	Similarity search	RAG
Multimodal	Multiple representations	Multimodal AI	Final Project

Feature Vector $\rightarrow$ Tensor $\rightarrow$ Graph $\rightarrow$ Embedding $\rightarrow$ Multimodal

Where Does Traditional Machine Learning Fit?



$$x = [x_1, x_2, \dots, x_d].$$

The model learns a relationship from the feature vector.

Running Case Study: Diabetes Prediction

Problem

Predict whether a person is likely to be diagnosed with diabetes from diagnostic measurements.

Target

$$y \in \{0, 1\}.$$

- ▶ 0: non-diabetic
- ▶ 1: diabetic

Input

$$x \in \mathbb{R}^8.$$

Pregnancies

Glucose

Blood Pressure

Skin Thickness

Insulin

BMI

Diabetes Pedigree

Age

Lee's Chapter 12 provides the running case through model evaluation and deployment.

From a Patient Record to a Feature Vector

Suppose one observation is represented by

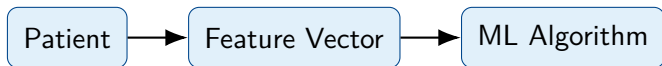
$$x = [p, g, bp, s, i, bmi, dpf, a].$$

More explicitly:

$$x = [\text{Pregnancies}, \text{Glucose}, \text{BloodPressure}, \text{SkinThickness}, \text{Insulin}, \text{BMI}, \text{Pedigree}, \text{Age}].$$

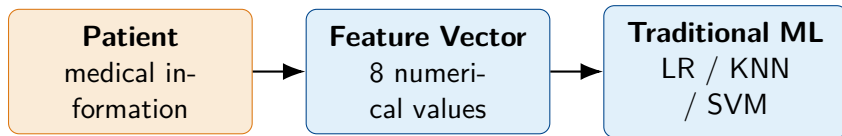
For N observations:

$$X = \begin{bmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_N^T \end{bmatrix} \in \mathbb{R}^{N \times d}, \quad d = 8.$$



Representation Is Not a Neutral Choice

Consider the same patient.



Question for Students

What information is preserved by this vector, and what information is not represented?

This question becomes increasingly important when we move to images, text, graphs, and multimodal data.

Formulating the Learning Problem

Given:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N,$$

we seek:

$$\hat{y} = f_{\theta}(x).$$

For classification:

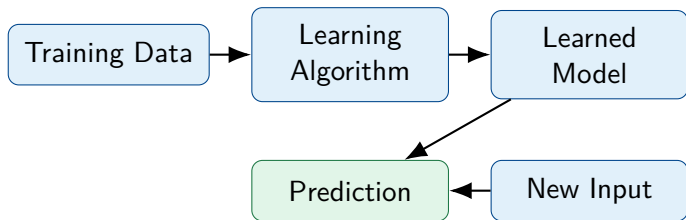
$$\hat{y} \in \{0, 1\}.$$

The learning process searches for:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(x_i), y_i).$$

Students must be able to explain every symbol.
--

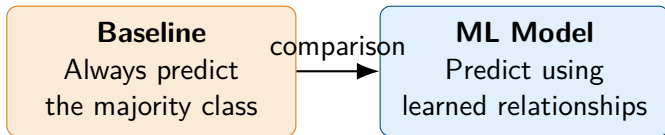
Supervised Learning: Learning from Examples



Training $\neq$ Testing

The test set estimates performance on unseen observations.

Why Do We Need a Baseline?



The question is not:

“Is my model accurate?”

but:

Does my model learn something useful beyond a simple baseline?

Traditional Models in the Diabetes Case

Model	Core idea	Representation
Logistic Regression	Linear decision function	Feature vector
KNN	Neighborhood similarity	Feature vector + distance
SVM	Maximum-margin separation	Feature vector / kernel space
Decision Tree	Recursive feature splits	Feature vector
Random Forest	Ensemble of trees	Feature vector

Key Idea

The algorithms receive essentially the same patient representation, but impose different assumptions and learning mechanisms.

Model 1: Logistic Regression

For binary classification:

$$z = w^T x + b$$

and

$$P(y = 1|x) = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Thus:

$$x \rightarrow w^T x + b \rightarrow \sigma(\cdot) \rightarrow P(y = 1|x).$$

Interpretation

- ▶ Each feature contributes through a learned weight.
- ▶ The output is interpreted as a probability-like score.
- ▶ A decision threshold converts the score into a class.

Question: What does a large positive or negative weight mean?

Model 2: k-Nearest Neighbors

KNN follows a very different idea.

A new observation is classified using nearby training observations.

For Euclidean distance:

$$d(x, x_i) = \sqrt{\sum_{j=1}^d (x_j - x_{ij})^2}.$$

Important consequence

Feature scale matters.

For example:

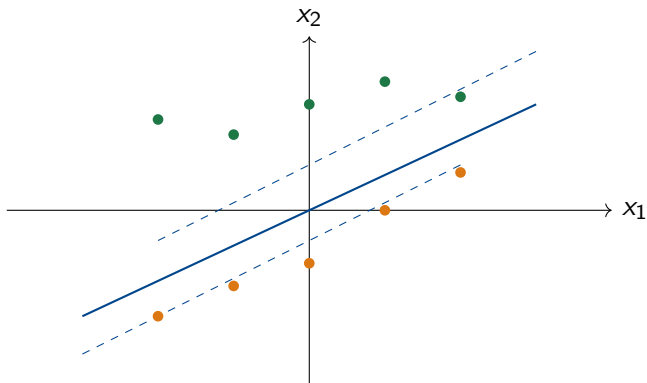
Glucose ~ 100 , Age ~ 40 .

A feature with a larger numerical scale can dominate distance.

Representation and preprocessing are connected.

Model 3: Support Vector Machine

SVM searches for a decision boundary with a large margin.



Good separation is not enough; the margin matters.

Model 4: Random Forest - Ensemble of Decision Trees

Core idea

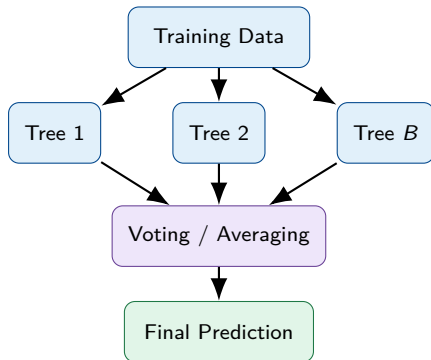
- ▶ Build many decision trees.
- ▶ Each tree sees a **bootstrap sample**.
- ▶ Each split considers a **random subset of features**.
- ▶ Combine all tree predictions.

$$\hat{y} = \text{Vote}(T_1(x), \dots, T_B(x))$$

For regression:

$$\hat{y} = \frac{1}{B} \sum_{b=1}^B T_b(x)$$

Why it works



Model 4: Random Forest - Ensemble of Decision Trees (cont)

Key benefit:

Many diverse trees $\Rightarrow$ more robust prediction

Random Forest = Bagging + Random Feature Selection + Ensemble Prediction

The Complete Assignment 01 Pipeline



Data → Representation → Model → Evidence → Decision

Looking Inside the Representation

One important analysis is examining relationships among features.

For features X_i and X_j :

$$\rho_{ij} = \text{corr}(X_i, X_j).$$

A correlation matrix can be represented as:

$$R = \begin{bmatrix} \rho_{11} & \rho_{12} & \cdots \\ \rho_{21} & \rho_{22} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}.$$

Why investigate correlation?

- ▶ understand relationships among variables;
- ▶ identify potentially redundant information;
- ▶ understand the dataset before modeling;
- ▶ motivate feature selection.

But: correlation does not imply causation.

From “Running Models” to Experiments

A weak experiment:

“I trained KNN, SVM, and Logistic Regression. KNN was best.”

A stronger experiment asks:

Why?

Students should investigate:

1. Does feature scaling change KNN?
2. Does changing k change KNN performance?
3. Does SVM respond differently to scaling?
4. Does feature selection change the result?
5. Do different metrics identify the same “best” model?

A parameter change should answer a question.
--

Evaluation: Accuracy Is Not Enough

For binary classification:

	Predicted +	Predicted -
Actual +	TP	FN
Actual -	FP	TN

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}.$$

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}.$$

$$F1 = \frac{2PR}{P + R}.$$

Students must explain which metric matters and why.

Model Selection Is a Decision

Suppose experiments produce:

Model	Accuracy	Precision	Recall	F1
Logistic Regression	—	—	—	—
KNN	—	—	—	—
SVM	—	—	—	—
Random Forest	—	—	—	—

The question is not simply:

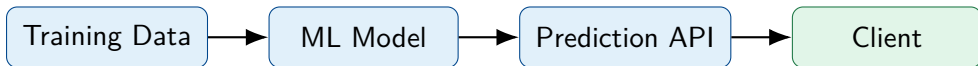
$$\max(\text{Accuracy})$$

Instead ask:

- ▶ Which model generalizes well?
- ▶ Which metrics are important?
- ▶ Is the model interpretable?
- ▶ What is its computational cost?
- ▶ Can it be reliably deployed?

From a Model to an Intelligent Application

A trained model is not yet a complete intelligent system.



Model + Interface + Data Processing + Prediction Service = Application

A deployed intelligent system may additionally require:

state + memory + feedback + monitoring.

Model Persistence

Once a model is trained, it can be saved for later use.

Training → Saved Model → Prediction.

Example:

```
import pickle

with open("model.pkl", "wb") as f:
    pickle.dump(model, f)
```

Then:

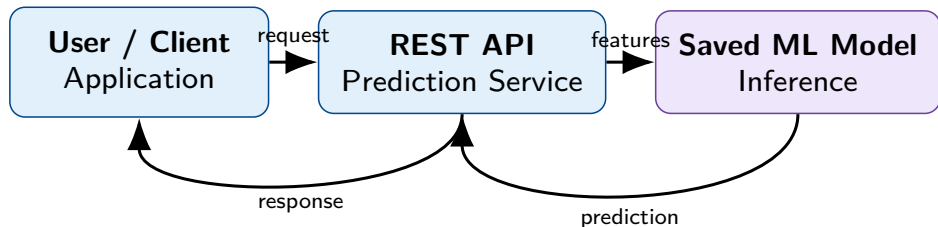
```
with open("model.pkl", "rb") as f:
    model = pickle.load(f)

prediction = model.predict(X_new)
```

Key Idea

New input must be represented and preprocessed in the same way as the training data.

Deployment: Model as a Service



Application does not need to know how the model learns.

It needs a well-defined prediction interface.

Assignment 01: What Must Students Deliver?

Foundation

1. Define the problem.
2. Define X and y .
3. Explain the representation.
4. Explain selected algorithms.
5. Explain assumptions and limitations.

Development

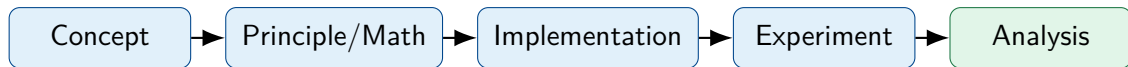
1. Use a real dataset.
2. Build at least 4 models.
3. Establish a baseline.
4. Conduct 3 experiments.
5. Evaluate with appropriate metrics.
6. Select a final model.
7. Build a small application.

Notebook + Report + Application + Explanation

Required Notebook Structure

1. Problem Definition
2. Dataset Source
3. Dataset Description
4. Data Representation
5. Feature Analysis
6. Data Exploration
7. Train/Test Split
8. Baseline
9. Model 1
10. Model 2
11. Model 3
12. Model 4
13. Evaluation
14. Model Comparison

What Makes This an Academic Assignment?



A good submission answers:

1. **What did you do?**
2. **Why did you do it?**
3. **What did you learn from the result?**

The Limitation of Traditional ML

Traditional pipeline:

Raw Data $\rightarrow$ Human-designed Features $\rightarrow$ ML Model

But consider an image.

A raw image contains:

$$H \times W \times C$$

values.

Can we manually design all useful features?

Probably not.

This motivates:

Representation Learning

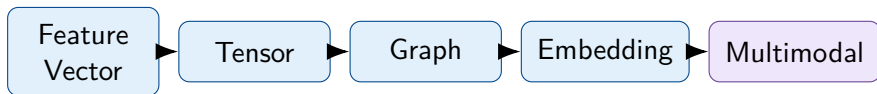
Deep learning attempts to learn useful representations from data.

Traditional ML vs Deep Learning

Traditional ML	Deep Learning
Human-designed features	Learned representations
Usually fixed-dimensional vectors	Vectors, tensors, sequences
Model learns prediction relationship	Model can learn representation + prediction
Often effective on structured data	Often effective on complex unstructured data
Usually smaller models	Potentially much larger models

Assignment 01: feature engineering → Later: representation learning

From Feature Vectors to Modern AI



- ▶ Feature vectors → traditional ML
- ▶ Tensors → deep learning
- ▶ Graphs → GNN / knowledge graphs
- ▶ Embeddings → semantic retrieval / RAG
- ▶ Multiple representations → multimodal AI

One course — one conceptual progression.

Why Will We Study Embeddings and RAG?

Traditional ML:

$$x \rightarrow f_{\theta}(x) \rightarrow y.$$

RAG introduces another operation:

Represent $\rightarrow$ Retrieve $\rightarrow$ Generate

For documents:

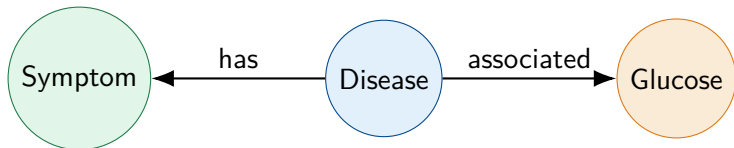
Document $\rightarrow$ Embedding $\rightarrow$ Similarity Search $\rightarrow$ Relevant Context $\rightarrow$ LLM.

The conceptual connection is:

Good intelligence requires an appropriate representation.

Why Will We Study Knowledge Graphs?

Some information is naturally relational.



A feature vector does not naturally express:

Entity $\rightarrow$ Relation $\rightarrow$ Entity.

This motivates graph-based representations and knowledge graphs.

The Five-Assignment Learning Journey

Assignment	Representation	Main learning
A1	Feature vectors	Traditional ML + deployment
A2	Better data representation	Preprocessing + visualization + metrics
A3	Learned representations	Neural networks + PyTorch
A4	Tensors + graphs	CNN + Knowledge Graph
A5	Embeddings + context	RAG + conversational systems
Project	Integrated representations	Complete intelligent system

Each assignment adds a new capability.

Assignment 01 Challenge

Do not simply reproduce Lee's diabetes example.

Students use the diabetes case to learn the workflow:

Problem → Representation → Algorithms → Experiments → Evaluation → Application.

Then apply the same methodology to a **different real dataset**.

**The diabetes case is the learning scaffold.
The student's dataset is the actual assignment.**

Assignment 01: Student Checklist

Before submission, verify:

- ☐ I can define the prediction problem.
- ☐ I can explain every input feature.
- ☐ I can describe the representation mathematically.
- ☐ I can explain why the task is classification/regression.
- ☐ I implemented a baseline.
- ☐ I trained at least four models.
- ☐ I performed at least three controlled experiments.
- ☐ I used appropriate evaluation metrics.
- ☐ I compared the models fairly.
- ☐ I can explain why I selected the final model.
- ☐ I built a small application.
- ☐ I can explain every important part of my notebook.

Reflection: From Model to System

What is your data?

How do you represent it?

**What can your model learn
from that representation?**

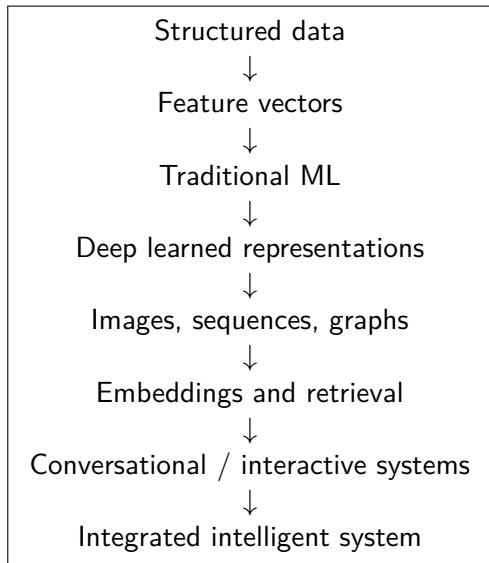
How do your experiments demonstrate that?

How does the model become an application?

**How does the application in-
teract with its environment?**

Understand → Represent → Implement → Experiment → Apply

The Journey of Intelligent System Development



The Big Picture

The course can be summarized by five questions:

1. **What is intelligence?**
2. **What should the system represent?**
3. **How can it learn or reason?**
4. **How does it decide and act?**
5. **How do we develop, evaluate, deploy, and improve it?**

Intelligence → Representation → Learning / Reasoning → Decision → Action → Feedback

This is the conceptual foundation of the course.

References

1. Wei-Meng Lee, *Python®Machine Learning*, First Edition, Wiley, 2019.
2. Wei-Meng Lee, "Chapter 12: Deploying Machine Learning Models."
3. Chapter 12 case study: Diabetes prediction using diagnostic measurements, followed by model comparison, model persistence, REST deployment, and a client application.
4. Scikit-learn documentation: machine learning algorithms and model evaluation.
5. Russell, S. and Norvig, P., *Artificial Intelligence: A Modern Approach*.
6. Goodfellow, I., Bengio, Y., and Courville, A., *Deep Learning*.
7. Vaswani, A. et al., "Attention Is All You Need," 2017.

Read → Understand → Reproduce → Modify → Experiment → Develop